Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Кафедра прикладної математики

Звіт
про виконання лабораторної роботи №4
з курсу «Основи інженерії даних»

Виконала:
студентка групи ПМІД-11
Мазур Д. М.

Перевірив:
Алєксєєв В. І.

Львів — 2026

Тема: Розподілена обробка даних та паралелізм з Apache Spark.

Мета роботи: Оволодіти концепціями розподілених обчислень, зосереджуючись на тому, як розподіл даних та операції перетасування (shuffle) впливають на продуктивність і масштабованість паралельних алгоритмів. Проаналізувати продуктивність паралельного алгоритму за допомогою закону Амдала та обміркувати, як обробляти асиметрію даних (data skew).

Постановка задачі: Проаналізувати великий набір даних лог-файлу, щоб обчислити середню тривалість сеансу користувача.

Завдання до роботи:

1. Підготувати набір даних Parquet зі схемою (`user_id`: STRING, `session_id`: STRING, `event_timestamp`: LONG, мілісекунди).
2. Налаштувати Spark для роботи на кластері з X виконавцями та Y ядрами на виконавця.
3. Реалізувати алгоритм 1 «наївне перетасування» (налаштування за замовчуванням), виміряти час і метрики перетасування.
4. Реалізувати алгоритм 2 «оптимізоване перетасування» з явним розподілом даних перед агрегацією.
5. Оцінити максимально досяжне прискорення за законом Амдала на основі виміряного часу.
6. Проаналізувати, як виявити та пом'якшити асиметрію даних, якщо один користувач становить 90% усіх сеансів.

Хід виконання роботи:

1. **Середовище та налаштування.** Використано PySpark 4.0.4 (Java 17) у локальному режимі. Режим `local[Y]` відповідає одному виконавцю (X = 1), що працює в JVM драйвера, з Y = 4 ядрами, тобто 4 паралельними задачами. На реальному кластері ці значення задаються параметрами `spark.executor.instances = X` і `spark.executor.cores = Y`, а загальна кількість паралельних задач дорівнює X×Y. Пам'ять драйвера становить 4 GB. Решта параметрів за замовчуванням: `spark.sql.shuffle.partitions = 200`, увімкнено Adaptive Query Execution (AQE).
2. **Підготовка даних.** Скрипт `lab4_db/generate_data.py` генерує лог засобами самого Spark (`spark.range`, без створення даних у Python). Згенеровано 400 000 сесій для 50 000 користувачів. Кожна сесія належить одному користувачу і містить від 2 до 20 подій у межах до 2 годин. Разом вийшло 4 397 801 подія. Події записано у 16 файлів Parquet із round-robin перемішуванням, тому події однієї сесії розкидані по різних файлах, як у реальних логах. Другий набір `logs_skewed.parquet` має таку саму структуру, але 90% сесій у ньому належать одному користувачу `user_whale`.
3. **Алгоритм розрахунку.** Кроки 2–4 завдання реалізовано однією функцією: групування за (`user_id`, `session_id`), тривалість як MAX−MIN мітки часу, потім загальне середнє:

```
def session_durations(df):
    return df.groupBy("user_id", "session_id").agg(
        (F.max("event_timestamp") - F.min("event_timestamp"))
        .alias("duration_ms"))

def average_session_duration(df):
    return session_durations(df).agg(F.avg("duration_ms")).collect()[0][0]
```

Вхідні дані попередньо кешуються в пам'яті (`df.cache().count()`), тому читання з диска не входить у виміряний час. Кожен варіант виконується раз для прогріву, потім тричі, і береться медіана. Метрики перетасування (обсяг shuffle write/read, кількість stages і tasks) зчитуються з REST API Spark UI для групи задач кожного запуску, а кількість операцій Exchange — з фізичного плану запиту.

4. **Порівняння стратегій розподілу.** Порівняно три варіанти:

   - *Стратегія A, «високе перетасування»* (алгоритм 1): `df.repartition(200)` без ключа перед `groupBy`. Round-robin перемішує *всі* сирі події, після чого `groupBy` виконує ще одне перетасування за ключем;
   - *Default*: просто `groupBy` з налаштуваннями за замовчуванням, для довідки;
   - *Стратегія B, «низьке перетасування»* (алгоритм 2): `df.repartition(4, 'user_id', 'session_id')`. Це hash-розподіл за ключем групування, кількість партицій дорівнює кількості ядер. Дані вже розподілені за потрібним ключем, тому `groupBy` не потребує другого Exchange.

Табл. 1: Порівняння стратегій розподілу (4 ядра, 4 397 801 подія)

Стратегія	Час, с	Shuffle, MB	Записів	Exchange	Tasks
A: repartition(200)	3.66	248.3	8 659 076	2	209
Default groupBy	0.55	43.7	1 411 222	1	9
B: repartition(4, ключі)	0.53	97.7	4 397 805	1	9

Усі стратегії дали однакову середню тривалість сесії: 2 787 661 мс $\approx$ 46.5 хв. Стратегія B виявилася в **6.8 раза швидшою** за стратегію A і перетасувала в 2.5 раза менше даних (97.7 проти 248.3 MB). У стратегії A кожна подія проходить через мережу двічі (8.66 млн записів у shuffle на 4.40 млн подій), а 200 дрібних задач додають накладні витрати на планування.

Варіант Default записав у shuffle найменше даних (43.7 MB) завдяки *частковій агрегації* (map-side combine): MAX і MIN обчислюються ще до перетасування. Тому явний `repartition` за ключем виправданий, коли розподіл використовують кілька подальших операцій або потрібен контроль кількості партицій.

5. **Моделювання продуктивності за законом Амдала.** Стратегію B виміряно на 1, 2, 4 і 8 ядрах (кількість партицій дорівнює кількості ядер). Закон Амдала описує час виконання на n ядрах як суму послідовної та паралельної частин:

$$T(n) = T_{\text{seq}} + \frac{T_{\text{par}}}{n}, \qquad S(n) = \frac{T(1)}{T(n)} = \frac{1}{(1-p) + p/n}, \qquad S_{\max} = \lim_{n\to\infty} S(n) = \frac{1}{1-p},$$

де $p = T_{\text{par}}/(T_{\text{seq}} + T_{\text{par}})$ — паралельна частка. Модель лінійна відносно $1/n$, тому T_{seq} і T_{par} знайдено методом найменших квадратів за чотирма вимірюваннями:

$$T(n) = 0.157 + \frac{1.618}{n}\ \text{с}, \qquad p = \frac{1.618}{0.157 + 1.618} = 0.912, \qquad S_{\max} = \frac{1}{1 - 0.912} \approx 11.3.$$

Додатково обчислено метрику Карпа–Флатта $e = \dfrac{1/S - 1/n}{1 - 1/n}$, тобто експериментально визначену послідовну частку (табл. 2, рис. 1).

Табл. 2: Масштабування стратегії B за кількістю ядер

n	$T(n)$, с	S виміряне	S за Амдалом	Ефективність S/n	Карп–Флатт e
1	1.761	1.00	1.00	1.00	–
2	1.007	1.75	1.84	0.87	0.144
4	0.533	3.30	3.16	0.83	0.070
8	0.360	4.89	4.94	0.61	0.091

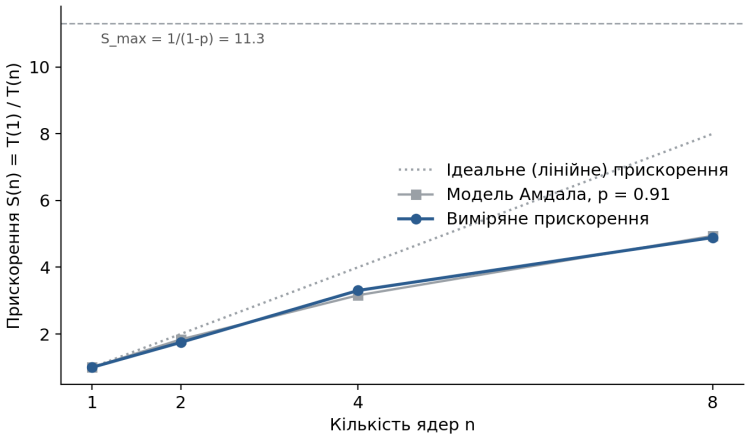


Рис. 1: Виміряне прискорення порівняно з моделлю Амдала та ідеальним лінійним прискоренням

Послідовні компоненти, які не масштабуються з ядрами: планування запиту та кодогенерація на драйвері, планування задач, фінальна агрегація AVG в одній партиції, `collect()` результату на драйвер, бар'єри синхронізації між stages. *Паралельні компоненти*: сканування кешованих партицій, часткова агрегація MAX/MIN, запис і читання shuffle-блоків, агрегація за ключем. Модель узгоджується з вимірюваннями з точністю до 5%. Оцінка p чутлива до шуму: в окремих запусках вона становила 0.76–0.91, тому $S_{\max}$ слід розглядати як орієнтир (від 4 до 11).

6. **Аналіз асиметрії даних (data skew).** Для набору `logs_skewed.parquet` користувач `user_whale` має 360 040 із 400 000 сесій (90.0%), а наступні за ним користувачі — лише по 6 сесій. *Виявлення* перекосу виконано двома способами: розподілом кількості сесій за `user_id` (top-N і частка лідера) та розміром кожної партиції після розподілу (`spark_partition_id`). У Spark UI перекіс видно як задачу-«відстаючу» (straggler), тривалість і Shuffle Read якої значно більші за медіану по stage. Порівняно три варіанти розподілу (табл. 3, рис. 2).

Табл. 3: Вплив перекосу та способів його пом'якшення (4 партиції)

Варіант розподілу	Найбільша партиція	max / медіана	Час, с
Наївний: за `user_id`	92.4%	36.6	1.21
Fix 1: за (`user_id`, `session_id`)	25.1%	1.0	0.48
Fix 2: salting, 32 значення солі	39.2%	1.6	0.71

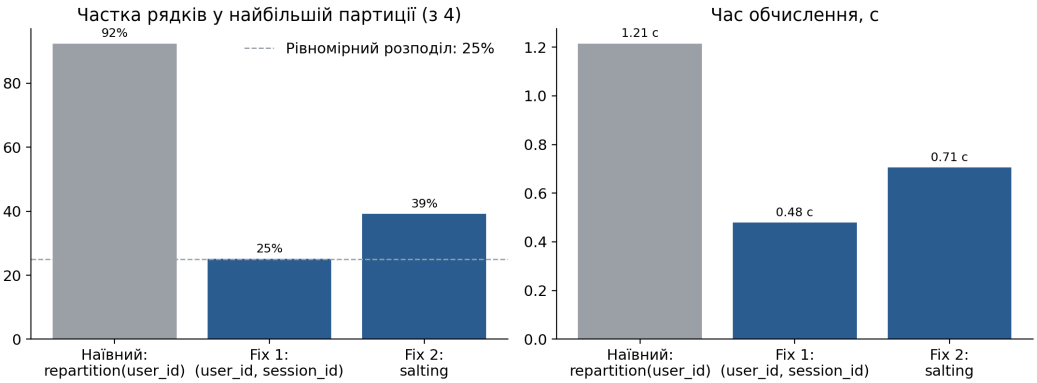


Рис. 2: Розподіл рядків між партиціями та час обчислення для перекошених даних

Якщо розподіляти дані лише за `user_id`, 92% рядків потрапляють в одну партицію: одне ядро виконує майже всю роботу, а решта простоюють. За Амдалом це рівнозначно збільшенню послідовної частки до ≈ 0.9. Тому час зростає у 2.5 раза порівняно зі збалансованим варіантом. Способи пом'якшення:

- *складений ключ* (`user_id`, `session_id`). Він і так є ключем групування, тож розподіл стає рівномірним (25.1%, тобто ідеальні 25%) без зміни логіки;
- *salting*: до «гарячого» ключа додається сіль, тут hash(`session_id`) mod 32. Сіль обчислюється з `session_id`, тому всі події однієї сесії отримують однакову сіль, і агрегація MAX−MIN залишається коректною за один крок. Для метрик на рівні користувача застосовується дво-етапна агрегація: спочатку за (`user_id`, `salt`), потім за `user_id`;
- для з'єднань: *AQE* (`spark.sql.adaptive.skewJoin.enabled`), окрема обробка «гарячих» ключів, broadcast малих таблиць.

Висновки: У ході виконання лабораторної роботи №4 засобами Apache Spark обчислено середню тривалість сесії ($\approx$ 46.5 хв) для логу з 4.4 млн подій. Стратегія з примусовим round-robin перетасуванням (`repartition(200)`) виявилася в 6.8 раза повільнішою за розподіл за ключем групування з кількістю партицій, що дорівнює кількості ядер. Вона передає кожну подію мережею двічі (248 проти 98 MB shuffle) і створює 200 дрібних задач. Закон Амдала дав паралельну частку $p \approx 0.91$ і граничне прискорення $S_{\max} \approx 11$; вже на 8 ядрах ефективність знижується до 0.61. Перекіс, коли один користувач має 90% сесій, видно за розміром партицій і задачами-«відстаючими». Розподіл лише за `user_id` збирає 92% даних в одній партиції та сповільнює обчислення в 2.5 раза, а розподіл за складеним ключем або salting усувають цю проблему. На реальному кластері, де перетасування йде через мережу, ці ефекти виражені ще сильніше.